CAPACITACIÓN QA AUTOMATION · PARTE 2 DE 2

MANUAL DE INSTALACIÓN

Automatización web con Playwright, TypeScript y Cucumber

De cero a un escenario BDD que abre un navegador real, inicia sesión y verifica el resultado. Para Windows, macOS y Linux, con un proyecto base que ya fue ejecutado en verde.

Node.js 24
LTS · motor de JavaScript

TypeScript
JavaScript con tipos

Playwright 1.64
controla el navegador

Cucumber 13
escenarios en Gherkin

01 Antes de empezar: qué vas a instalar
02 Paso 1 · Node.js y npm
03 Paso 2 · VS Code y sus extensiones
04 Paso 3 · Crear el proyecto
05 Paso 4 · Configuración
06 Paso 5 · World y hooks
07 Paso 6 · Feature, Page Object y steps
08 Paso 7 · Ejecutar y leer el reporte
09 Errores típicos y su solución
10 Checklist final

Instructor: **Giancarlo Palomino** · QA Automation Engineer

Windows · macOS · Linux · v1.0 · octubre 2026

MAPA DE LA INSTALACIÓN

Qué vas a instalar y para qué sirve cada pieza

Imagina un robot que usa la web como lo haría una persona. Node.js es su energía, npm la tienda donde compra herramientas, TypeScript el idioma con corrector ortográfico, Playwright sus manos y ojos, y Cucumber el guion que le dice qué hacer, escrito para que negocio también lo lea.

1 · MOTOR
Node.js + npm
ejecuta JS e instala paquetes

→

2 · EDITOR
VS Code
el mismo de la parte 1

→

3 · PAQUETES
npm install
Playwright, Cucumber, TS

→

4 · NAVEGADORES
playwright install
Chromium para las pruebas

PIEZA	PARA QUÉ SIRVE	VERSIÓN PROBADA	CÓMO LLEGA
Node.js	Ejecuta JavaScript fuera del navegador. Usa una versión **LTS** (soporte largo): 24 o 22.	`24` / `22`	Instalador
npm	Viene con Node. Descarga los paquetes listados en `package.json` a `node_modules`.	con Node	Instalador
VS Code	Editor con terminal integrada y extensiones para Cucumber y Playwright.	reciente	Instalador
@playwright/test	Abre navegadores, hace clic, escribe y verifica con esperas automáticas.	`1.64.1`	npm
@cucumber/cucumber	Lee los `.feature` en Gherkin y ejecuta el código de cada paso.	`13.3.0`	npm
typescript	Revisa los tipos antes de ejecutar: detecta errores al escribir, no en la demo.	`7.0.2`	npm
tsx	Permite que Cucumber cargue archivos `.ts` directamente, sin compilar a mano.	`4.23`	npm
cross-env	Define variables de entorno con la misma sintaxis en Windows, macOS y Linux.	`10.1.0`	npm

REQUISITOS DEL EQUIPO
Windows 10/11, macOS 13+ o Linux (Ubuntu 22.04+, Debian 12+, Fedora 39+) · 8 GB de RAM · 2 GB libres para navegadores · acceso a `registry.npmjs.org` y `playwright.download.prss.microsoft.com`.

PROYECTO VERIFICADO
Todo el código de este manual se ejecutó el 26-10-2026 contra `saucedemo.com`: 3 escenarios, 15 pasos en verde, en 1,2 segundos tras la primera carga.

SI YA HICISTE LA PARTE 1
Ya tienes VS Code: solo te faltan Node.js (Paso 1) y dos extensiones más (Paso 2). No necesitas Java para esta parte.

MOTOR

1 Instalar Node.js LTS (24 o 22)

Cucumber 13 exige Node **22 o 24**. Recomendamos un **gestor de versiones** (nvm, fnm): permite tener varias versiones y cambiar entre ellas por proyecto, sin pedir permisos de administrador.

Windows 10 / 11

OPCIÓN A · INSTALADOR

```
# PowerShell
winget install
OpenJS.NodeJS.LTS
```

O el `.msi` de `nodejs.org`: deja marcada la opción *Add to PATH*.

OPCIÓN B · GESTOR FNM

```
winget install Schniz.fnm
fnm install --lts
fnm use lts-latest
```

fnm pide una línea en tu perfil de PowerShell; la indica al instalar.

VERIFICAR (TERMINAL NUEVA)

```
node -v
npm -v
```

macOS INTEL / APPLE SILICON

RECOMENDADO · NVM

```
curl -o-
https://raw.githubusercontent.
com/nvm-
sh/nvm/v0.40.3/install.sh |
bash
```

Cierra y abre la terminal; luego:

```
nvm install --lts
nvm alias default 'lts/*'
```

ALTERNATIVA · HOMEBREW

```
brew install node@24
```

Brew avisa si debes agregarlo al `PATH` en `~/.zshrc`.

VERIFICAR

```
node -v
npm -v
```

Linux UBUNTU · DEBIAN · FEDORA

RECOMENDADO · NVM

```
curl -o-
https://raw.githubusercontent.
com/nvm-
sh/nvm/v0.40.3/install.sh |
bash
source ~/.bashrc
nvm install --lts
```

EVITA APT INSTALL NODEJS

Los repositorios de muchas distribuciones traen versiones viejas (18 o 20) que Cucumber 13 rechaza.

¿Necesitas paquete del sistema? Usa el repositorio oficial de NodeSource.

VERIFICAR

```
node -v
npm -v
```

```
# Resultado esperado (cualquier sistema)
$ node -v
v22.19.0      # o v24.x: ambos sirven
$ npm -v
11.11.1
```

SOLO WINDOWS: "LA EJECUCIÓN DE SCRIPTS ESTÁ DESHABILITADA"

Si `npm -v` falla en PowerShell con ese mensaje, ejecuta una sola vez: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` y responde **S**.

MESA DE TRABAJO

2 VS Code y sus extensiones

Es el mismo editor de la parte 1. Si ya lo tienes, salta directo a las extensiones.

1 · Instalar VS Code

Windows — WINGET

```
winget install Microsoft.VisualStudioCode
```

macOS — HOMEBREW

```
brew install --cask visual-studio-code
```

Linux — SNAP · O .DEB/.RPM

```
sudo snap install code --classic
```

2 · Extensiones (igual en los tres sistemas)

Terminal — INSTALAR

```
code --install-extension CucumberOpen.cucumber-official
code --install-extension ms-playwright.playwright
```

- **Cucumber** (CucumberOpen): colores y autocompletado en los `.feature`; con `F12` saltas del paso a su función.
- **Playwright Test for VS Code** (Microsoft): incluye *Pick locator* para obtener selectores haciendo clic en la página, y la grabadora de acciones.

3 · Decirle a Cucumber dónde está todo

.vscode/settings.json — EN EL PROYECTO

```
{
  "cucumber.features": ["features/**/*.feature"],
  "cucumber.glue": ["src/steps/**/*.ts"]
}
```

¿POR QUÉ EL ARCHIVO SETTINGS.JSON?

Sin él, la extensión busca en rutas por defecto y marca todos los pasos como *"Undefined step"* aunque el código exista. Con él, autocompleta frases que ya tienen código y avisa cuando escribes una nueva.

LA EXTENSIÓN DE PLAYWRIGHT Y CUCUMBER

La vista **Testing** de esa extensión ejecuta pruebas de `@playwright/test`, no escenarios de Cucumber. En este proyecto la usamos para *Pick locator* y *Record new*; los escenarios se ejecutan con `npm test`.

ATAJOS ÚTILES (WINDOWS/LINUX · MACOS)

Ir del paso a su código: `F12`
Terminal integrada: `Ctrl` + `` ` `` · `^` + `` ` ``
Paleta de comandos: `Ctrl` + `Shift` + `P` · `⌘` + `⇧` + `P`
Formatear archivo: `Shift` + `Alt` + `F` · `⇧` + `⌥` + `F`

COMANDO "CODE" EN MACOS

Si la terminal no reconoce `code`, abre VS Code, pulsa `⌘` + `⇧` + `P` y ejecuta *Shell Command: Install 'code' command in PATH*.

ESQUELETO

3 Crear el proyecto e instalar paquetes

Estos comandos son **idénticos en Windows, macOS y Linux**. Ejecútalos uno por uno en la terminal integrada de VS Code, dentro de una carpeta vacía.

Terminal — PASO A PASO

```
# 1. Crear la carpeta y el package.json
mkdir e2e-bdd-playwright
cd e2e-bdd-playwright
npm init -y
npm pkg set type=module

# 2. Instalar las herramientas como dependencias de desarrollo
npm install -D @playwright/test @cucumber/cucumber typescript tsx @types/node cross-env

# 3. Descargar el navegador Chromium que usará Playwright
npx playwright install chromium
```

Windows — PASO 3

El comando de arriba basta. Descarga Chromium a `%USERPROFILE%\AppData\Local\ms-playwright`.

macOS — PASO 3

El comando de arriba basta. Descarga a `~/Library/Caches/ms-playwright`.

Linux — PASO 3

Agrega las librerías del sistema que necesita el navegador:

```
npx playwright install --with-deps chromium
```

Pide tu clave de `sudo`.

```
e2e-bdd-playwright/
├── package.json          # scripts y dependencias
├── tsconfig.json         # reglas de TypeScript
├── cucumber.js           # dónde buscar todo
├── .vscode/settings.json
├── features/
│   └── login.feature     # escenarios
└── src/
    ├── pages/
    │   └── LoginPage.ts      # Page Object
    ├── steps/
    │   └── login.steps.ts    # Gherkin → TS
    └── support/
        ├── world.ts          # estado
        └── hooks.ts          # antes/después
```

-D (DEVDEPENDENCIES)
Son herramientas de prueba: no forman parte de la aplicación que se entrega.

NODE_MODULES
Carpeta pesada que npm genera. Nunca se sube al repositorio: se regenera con `npm install`.

PROXY CORPORATIVO
Si la descarga de Chromium falla, define `HTTPS_PROXY` con la dirección del proxy antes del paso 3.

LAS REGLAS DEL JUEGO

4 package.json, tsconfig.json y cucumber.js

package.json — RESULTADO TRAS EL PASO 3 + SCRIPTS

```
{
  "name": "e2e-bdd-playwright",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "test": "cross-env NODE_OPTIONS=--import=tsx cucumber-js",
    "test:smoke": "cross-env NODE_OPTIONS=--import=tsx cucumber-js --tags @smoke",
    "test:headed": "cross-env HEADED=1 NODE_OPTIONS=--import=tsx cucumber-js",
    "typecheck": "tsc --noEmit"
  },
  "devDependencies": {
    "@cucumber/cucumber": "^13.3.0",
    "@playwright/test": "^1.64.1",
    "@types/node": "^26.6.5",
    "cross-env": "^10.1.0",
    "tsx": "^4.23.15",
    "typescript": "^7.0.2"
  }
}
```

tsconfig.json — TYPESCRIPT

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "types": ["node"]
  },
  "include": ["src", "features"]
}
```

cucumber.js — CUCUMBER

```
export default {
  paths: ['features/**/*.feature'],
  import: ['src/**/*.ts'],
  format: [
    'progress-bar',
    'html:reports/cucumber-report.html',
  ],
  formatOptions: { snippetInterface: 'async-
await' },
  language: 'es',
};
```

NODE_OPTIONS=--IMPORT=TSX
Le enseña a Node a leer `.ts`. Sin esto, Cucumber no entiende TypeScript.

CROSS-ENV
Hace que `HEADED=1` funcione igual en PowerShell, zsh y bash.

LANGUAGE: 'ES'
Gherkin en español por defecto: *Característica, Escenario, Dado…*

LAS PIEZAS QUE NO SE VEN

5 El World y los hooks

El **World** es la mochila de cada escenario: ahí viaja la página del navegador para que todos los pasos la usen con `this.page`. Los **hooks** preparan y limpian alrededor de cada escenario.

src/support/world.ts — ESTADO DEL ESCENARIO

```ts
import { World, IWorldOptions, setWorldConstructor } from '@cucumber/cucumber';
import { BrowserContext, Page } from '@playwright/test';

// El World es el contexto que comparten los steps de un mismo escenario.
export class CustomWorld extends World {
  context!: BrowserContext;
  page!: Page;

  constructor(options: IWorldOptions) {
    super(options);
  }
}

setWorldConstructor(CustomWorld);
```

src/support/hooks.ts — ANTES Y DESPUÉS

```ts
import { After, AfterAll, Before, BeforeAll, Status, setDefaultTimeout } from '@cucumber/cucumber';
import { Browser, chromium } from '@playwright/test';
import { CustomWorld } from './world';

setDefaultTimeout(30_000);

const BASE_URL = process.env.BASE_URL ?? 'https://www.saucedemo.com';
let browser: Browser;

BeforeAll(async () => {
  browser = await chromium.launch({ headless: process.env.HEADED !== '1' });
});

AfterAll(async () => {
  await browser.close();
});

// Contexto y página nuevos por escenario: ningún test hereda la sesión de otro.
Before(async function (this: CustomWorld) {
  this.context = await browser.newContext({ baseURL: BASE_URL });
  this.page = await this.context.newPage();
});

After(async function (this: CustomWorld, { result }) {
  if (result?.status === Status.FAILED) {
    this.attach(await this.page.screenshot(), 'image/png');
  }
  await this.context?.close();
});
```

El ciclo de vida, en orden

UNA VEZ — **BeforeAll** — abre el navegador → CADA ESCENARIO — **Before** — contexto limpio → CADA ESCENARIO — **Steps** — Dado · Cuando · Entonces → CADA ESCENARIO — **After** — captura si falló → UNA VEZ — **AfterAll** — cierra el navegador

LO QUE SÍ SE VE

6 El feature y el Page Object

El feature dice **qué** se prueba, en lenguaje de negocio. El Page Object guarda **cómo** se maneja la pantalla: si mañana cambia un selector, se corrige en un solo lugar y ningún escenario se entera.

features/login.feature — GHERKIN

```gherkin
# language: es
Característica: Inicio de sesión en SauceDemo
  Como comprador registrado
  Quiero iniciar sesión
  Para ver el catálogo de productos

  Antecedentes:
    Dado que estoy en la página de login

  @smoke
  Escenario: Login exitoso
    Cuando ingreso con el usuario "standard_user" y la clave "secret_sauce"
    Entonces veo el catálogo de productos

  @regresion
  Esquema del escenario: Login rechazado
    Cuando ingreso con el usuario "<usuario>" y la clave "<clave>"
    Entonces veo el mensaje de error "<mensaje>"

    Ejemplos:
      | usuario         | clave        | mensaje                                 |
      | locked_out_user | secret_sauce | Sorry, this user has been locked out |
      | standard_user   | incorrecta   | Username and password do not match   |
```

src/pages/LoginPage.ts — PAGE OBJECT

```typescript
import { Page } from '@playwright/test';

export class LoginPage {
  constructor(private readonly page: Page) {}

  async abrir() {
    await this.page.goto('/');
  }

  async ingresar(usuario: string, clave: string) {
    await this.page.locator('[data-test="username"]').fill(usuario);
    await this.page.locator('[data-test="password"]').fill(clave);
    await this.page.locator('[data-test="login-button"]').click();
  }

  mensajeError() {
    return this.page.locator('[data-test="error"]');
  }
}
```

SELECTORES DATA-TEST
Atributos puestos para pruebas: no cambian cuando diseño mueve un botón o cambia un color.

GOTO('/')
Ruta relativa: la URL base la pone el hook. Mismo código para QA, staging o producción.

SIN ESPERAS MANUALES
Playwright espera solo a que el campo exista y sea editable antes de escribir.

GHERKIN → TYPESCRIPT

6 Las step definitions

src/steps/login.steps.ts — STEPS

```ts
import { Given, When, Then } from '@cucumber/cucumber';
import { expect } from '@playwright/test';
import { CustomWorld } from '../support/world';
import { LoginPage } from '../pages/LoginPage';

Given('que estoy en la página de login',
  async function (this: CustomWorld) {
    await new LoginPage(this.page).abrir();
  });

When('ingreso con el usuario {string} y la clave {string}',
  async function (this: CustomWorld, usuario: string, clave: string) {
    await new LoginPage(this.page).ingresar(usuario, clave);
  });

Then('veo el catálogo de productos',
  async function (this: CustomWorld) {
    await expect(this.page.locator('[data-test="title"]'))
      .toHaveText('Products');
  });

Then('veo el mensaje de error {string}',
  async function (this: CustomWorld, mensaje: string) {
    await expect(new LoginPage(this.page).mensajeError())
      .toContainText(mensaje);
  });
```

Cómo encuentra Cucumber el código de cada paso

TEXTO DEL FEATURE
`ingreso con el usuario "standard_user" y la clave "secret_sauce"`

→

EXPRESIÓN DEL STEP
`ingreso con el usuario {string} y la clave {string}`

→

PARÁMETROS
`(usuario, clave)`

¿GIVEN EN INGLÉS SI EL FEATURE ESTÁ EN ESPAÑOL?

Sí: en código, `Given / When / Then` sirven para cualquier idioma. Cucumber une por el **texto** de la frase, no por la palabra clave: *"Dado que estoy…"* encuentra a `Given('que estoy…')`.

ASERCIONES WEB-FIRST

`await expect(locator).toHaveText(...)` reintenta hasta que el texto aparezca (5 s por defecto). Nunca uses `waitForTimeout`: hace las pruebas lentas e inestables.

LUZ VERDE

7 Ejecutar y leer el resultado

COMANDO (IGUAL EN LOS TRES SISTEMAS)	QUÉ HACE
`npm test`	Ejecuta todos los escenarios con el navegador oculto (*headless*): rápido, ideal para CI.
`npm run test:smoke`	Solo los escenarios con `@smoke`.
`npm run test:headed`	Muestra el navegador mientras trabaja. Ideal para aprender y depurar.
`npm test -- --tags "not @regresion"`	Cualquier opción de Cucumber va después de `--`.
`npm run typecheck`	Revisa los tipos de TypeScript sin ejecutar nada.

Así se ve una ejecución correcta

```
$ npm test

> e2e-bdd-playwright@1.0.0 test
> cross-env NODE_OPTIONS=--import=tsx cucumber-js

.................

2 hooks (2 passed)
3 scenarios (3 passed)
15 steps (15 passed)
0m 4.538s (0m 4.510s executing your code)
```

El reporte HTML

Abre `reports/cucumber-report.html` en el navegador: cada escenario con sus pasos en verde o rojo, y la captura de pantalla adjunta cuando algo falla (la toma el hook `After`).

Cambiar de ambiente

El hook lee `BASE_URL`. Con cross-env la sintaxis es la misma en todos lados: `npx cross-env BASE_URL=https://qa.miapp.com npm test`.

"CANNOT USE 'PROGRESS-BAR' FORMATTER… NOT A TTY"

No es un error: ocurre cuando la salida no va a una terminal interactiva (por ejemplo, en CI). Cucumber cambia solo a otro formato.

PRUEBA DE HUMO DEL ENTORNO

Si viste **3 scenarios (3 passed)**, tu equipo está listo: Node, los paquetes, el navegador y la conexión funcionan.

EJECUTAR EN LINUX SIN ESCRITORIO

En un servidor o WSL sin interfaz gráfica, `npm run test:headed` no puede abrir ventanas. Usa `npm test` (headless).

CUANDO ALGO SALE EN ROJO

Errores típicos y cómo resolverlos

El mensaje casi siempre dice exactamente qué falta. Léelo completo antes de buscar en internet.

```
npm : No se puede cargar el archivo C:\Program Files\nodejs\npm.ps1 porque la ejecución de scripts está deshabilitada en este sistema.
```

CAUSA · SOLO WINDOWS	SOLUCIÓN
La política de PowerShell bloquea scripts por defecto.	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, una sola vez. O usa la terminal *Command Prompt*.

```
browserType.launch: Executable doesn't exist at …/ms-playwright/chromium_headless_shell-1248/… Please run the following command to download new browsers: npx playwright install
```

CAUSA	SOLUCIÓN
Se instaló o actualizó `@playwright/test` pero no se descargó el navegador de esa versión.	`npx playwright install chromium` (en Linux, con `--with-deps`). Repítelo cada vez que actualices Playwright.

```
Host system is missing dependencies to run browsers. Please install them with: sudo npx playwright install-deps
```

CAUSA · SOLO LINUX	SOLUCIÓN
Faltan librerías gráficas del sistema (libnss3, libgbm…).	`npx playwright install --with-deps chromium` o `sudo npx playwright install-deps`.

```
SyntaxError [ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX]: TypeScript parameter property is not supported in strip-only mode
```

CAUSA	SOLUCIÓN
Ejecutaste `npx cucumber-js` directamente, sin `NODE_OPTIONS=--import=tsx`. Node intenta leer el TypeScript por su cuenta y no entiende todo.	Ejecuta siempre con `npm test` (el script ya incluye tsx), nunca `npx cucumber-js` a secas.

```
'HEADED' no se reconoce como un comando interno o externo · HEADED=1 : El término no se reconoce…
```

CAUSA · WINDOWS	SOLUCIÓN
Escribiste `HEADED=1 npm test`, sintaxis de macOS/Linux.	Usa `npm run test:headed`: el script usa `cross-env` y funciona igual en los tres sistemas.

```
Undefined. Implement with the following snippet: When('ingreso con el usuario {string} y la contraseña {string}', async function (string, string2) { … })
```

CAUSA
La frase del feature no coincide con ninguna expresión (una tilde, una palabra distinta) o el archivo de steps está fuera de `import` en `cucumber.js`.

SOLUCIÓN
Corrige la frase o copia el snippet propuesto en tu archivo de steps.

```
Multiple step definitions match: que estoy en la página de login — src/steps/login.steps.ts:6 / src/steps/carrito.steps.ts:9
```

CAUSA
La misma frase está definida dos veces (copiar y pegar entre archivos).

SOLUCIÓN
Deja una sola definición; los pasos comunes van a un archivo compartido, por ejemplo `comun.steps.ts`.

```
Error: function timed out, ensure the promise resolves within 5000 milliseconds
```

CAUSA
Cucumber da 5 segundos por paso; abrir el navegador o una página lenta tarda más.

SOLUCIÓN
`setDefaultTimeout(30_000)` en `hooks.ts` (ya incluido). Si un paso sigue excediéndolo, el problema está en el selector o la app.

```
TimeoutError: locator.fill: Timeout … exceeded. waiting for locator('[data-test="usuario"]')
```

CAUSA
El selector no existe en la página (nombre mal escrito, o la página cambió).

SOLUCIÓN
Obtén el selector real con *Pick locator* de la extensión de Playwright o con `npx playwright codegen saucedemo.com`.

```
npm error code ETIMEDOUT / SELF_SIGNED_CERT_IN_CHAIN · Error: getaddrinfo ENOTFOUND playwright.download.prss.microsoft.com
```

CAUSA
Red corporativa con proxy o inspección de certificados.

SOLUCIÓN
`npm config set proxy http://proxy:8080` y `https-proxy`; para Playwright, variable `HTTPS_PROXY`. Si es un certificado, pide el raíz a Mesa de Ayuda y usa `NODE_EXTRA_CA_CERTS`.

```
EBADENGINE Unsupported engine { package: '@cucumber/cucumber@13.3.0', required: { node: '22 || 24 || >=26' }, current: { node: 'v20.x' } }
```

CAUSA
Tu Node es más viejo que lo que exige Cucumber 13.

SOLUCIÓN
Instala Node 24 LTS (Paso 1) o, con nvm/fnm, `nvm install --lts`, y vuelve a ejecutar `npm install`.

ANTES DE LA PRIMERA CLASE

Checklist: ¿está todo listo?

Si todas las casillas están marcadas, llegas a la clase con el entorno funcionando y dedicamos el tiempo a aprender, no a instalar.

Entorno

- [ ] `node -v` responde v22 o v24.
- [ ] `npm -v` responde sin errores (en Windows, también en PowerShell).
- [ ] VS Code tiene las extensiones **Cucumber** y **Playwright Test**.
- [ ] `F12` sobre un paso del feature lleva a su función en `login.steps.ts`.
- [ ] En Linux: ejecutaste `npx playwright install --with-deps chromium`.

Proyecto

- [ ] `npm test` muestra **3 scenarios (3 passed)**.
- [ ] `npm run test:headed` abre el navegador y lo ves iniciar sesión.
- [ ] `npm run typecheck` termina sin errores.
- [ ] Existe `reports/cucumber-report.html` y se abre en el navegador.
- [ ] Cambiaste `'Products'` por `'Productos'`, viste el fallo con su captura en el reporte y lo regresaste.

¿POR QUÉ ROMPER A PROPÓSITO?
La primera vez que veas un fallo debe ser en un entorno controlado. Reconocer un error de aserción, un selector que no existe y un paso sin definir te ahorra horas cuando ocurra de verdad.

Lo que viene en la presentación

CONCEPTOS
Cómo piensa Playwright: localizadores, esperas automáticas y aserciones web-first.

DISEÑO
World, hooks y Page Objects: por qué separar el qué (Gherkin) del cómo (código).

EJERCICIOS
Cinco retos progresivos sobre SauceDemo, terminando con una caza de bugs con el usuario `problem_user`.

PROYECTO BASE
El proyecto de este manual se entrega como `e2e-bdd-playwright.zip`. Descomprímelo, ejecuta `npm install`, `npx playwright install chromium` y `npm test`: si sale verde, tu instalación está validada.